



تمرین برنامه نویسی شماره صفر

برنامه نویسی موازی

آشنایی با پروژه درس

دکتر سعید صفری

طراحان: امیرحسین راحتی، هاتف رضائی، سپهر جمالی

نیمسال اول سال تحصیلی 05-14

اهداف پروژه

این پروژه با هدف آشنایی با قابلیت‌ها و مفاهیم سطح پایین پردازنده در برنامه‌نویسی موازی و همچنین پیاده‌سازی گام‌پایه‌ی شبیه‌سازی حرکت ذرات طراحی شده است. مراحل اصلی این پروژه عبارتند از:

1. استخراج و بررسی ویژگی‌های سخت‌افزاری و مجموعه‌دستورالعمل‌های پردازنده از طریق دستور `CPUID`.
2. تحلیل و تفسیر قالب‌های مختلف داده بر روی حافظه‌ی ۱۲۸ بیتی با به‌کارگیری ساختار `union`.
3. پیاده‌سازی نسخه اولیه و اسکالر شبیه‌سازی ذرات به عنوان معیار و مرجع مقایسه جهت سنجش کارایی در فازهای موازی‌سازی آتی.

قسمت اول

آشنایی با پردازنده و دستورات سطح پایین

در این قسمت هدف آن است که با برخی اطلاعات سخت‌افزاری پردازنده و نحوه‌ی دسترسی به آن‌ها از طریق دستور `CPUID` آشنا شوید. همچنین نحوه‌ی تفسیر یک داده‌ی ۱۲۸ بیتی در قالب انواع داده‌ی مختلف را بررسی خواهید کرد.

۱. استخراج اطلاعات پردازنده با دستور CPUID

دستور `CPUID` یکی از دستورهای سطح پایین معماری x86 است که برای دریافت اطلاعات مربوط به پردازنده و قابلیت‌های آن مورد استفاده قرار می‌گیرد. با استفاده از این دستور، اطلاعات زیر را از پردازنده‌ی سیستم خود استخراج کنید:

1. نام کامل پردازنده
   نام پردازنده را استخراج کرده و اطلاعات مربوط به مدل و نسل آن را مشخص کنید.
2. تعداد هسته‌های پردازنده
   تعداد هسته‌های فیزیکی و منطقی پردازنده‌ی خود را مشخص کنید.
   همچنین با استفاده از اطلاعات مربوط به `CPUID` بررسی کنید که آیا پردازنده از قابلیت Hyper-Threading پشتیبانی می‌کند یا خیر.

3. بررسی Hyper-Threading

در صورت امکان، وضعیت Hyper-Threading پردازنده را از طریق BIOS تغییر دهید. در صورتی که این قابلیت فعال است، آن را غیرفعال کرده و در غیر این صورت آن را فعال کنید.
پس از تغییر، مجددا تعداد هسته‌های فیزیکی و منطقی را بررسی کنید.

4. فرکانس پردازنده

حداکثر فرکانس کاری پردازنده را با استفاده از اطلاعات مربوط به `CPUID` استخراج کنید.
سپس پردازنده‌ی خود را در وب‌سایت رسمی شرکت سازنده پیدا کرده و مقدار به‌دست‌آمده را با مشخصات اعلام‌شده توسط سازنده مقایسه کنید.

5. بررسی پشتیبانی از مجموعه‌دستورالعمل‌های SIMD

با استفاده از `CPUID` بررسی کنید که پردازنده‌ی شما از مجموعه‌دستورالعمل‌های زیر پشتیبانی می‌کند یا خیر:

a. SSE
b. SSE2
c. SSE3
d. SSE4
e. AVX
f. AVX2

6. اطلاعات Cache و TLB

با استفاده از اطلاعات ارائه‌شده توسط `CPUID`، اطلاعات مربوط به Cache و TLB پردازنده را استخراج کنید.

۲. تفسیر داده‌ی ۱۲۸ بیتی با استفاده از Union

در این بخش هدف آن است که با نحوه‌ی نمایش داده‌ها در حافظه و تفسیر یک بخش یکسان از حافظه با انواع داده‌ی مختلف آشنا شوید. تابعی بنویسید که یک داده‌ی ۱۲۸ بیتی را دریافت کرده و بر اساس نوع داده‌ی مشخص‌شده، محتوای آن را چاپ کند. نوع داده می‌تواند یکی از موارد زیر باشد:

```
None
i8   : چاپ ۱۶ عدد با علامت ۸ بیتی
u16  : چاپ ۸ عدد بدون علامت ۱۶ بیتی
i16  : چاپ ۸ عدد با علامت ۱۶ بیتی
```

```
چاپ ۴ عدد بدون علامت ۳۲ بیتی : u32
چاپ ۴ عدد با علامت ۳۲ بیتی : i32
چاپ ۲ عدد بدون علامت ۶۴ بیتی : u64
چاپ ۲ عدد با علامت ۶۴ بیتی : i64
```

برای پیاده‌سازی این بخش از ساختار `union` استفاده کنید تا یک داده‌ی ۱۲۸ بیتی از دیدگاه انواع داده‌ی مختلف قابل مشاهده باشد. در زمان ارائه باید بتوانید نحوه‌ی عملکرد این تابع و دلیل تفاوت خروجی در انواع داده‌ی مختلف را توضیح دهید.

قسمت دوم

پیاده‌سازی نسخه Scalar شبیه‌سازی

در این قسمت یک شبیه‌سازی از حرکت و برخورد ذرات در اختیار شما قرار گرفته است. هدف این بخش، درک ساختار شبیه‌سازی و پیاده‌سازی نسخه‌ی Scalar آن است.

نسخه‌ی Scalar، نسخه‌ی پایه‌ی شبیه‌سازی محسوب می‌شود و در ادامه‌ی پروژه برای مقایسه‌ی کارایی روش‌های مختلف موازی‌سازی مورد استفاده قرار خواهد گرفت.

معرفی شبیه‌سازی

در این شبیه‌سازی، با کلیک روی صفحه مجموعه‌ای از ذرات ایجاد می‌شود که حرکت آن‌ها تحت تأثیر نیروها و قوانین مختلف فیزیکی قرار دارد. حرکت هر ذره تحت تأثیر عوامل زیر محاسبه می‌شود:

1. گرانش N-body

   هر ذره تحت تاثیر نیروی گرانشی سایر ذرات قرار دارد. بنابراین برای محاسبه‌ی نیروی وارد شده به هر ذره، باید اثر تمام ذرات دیگر بررسی شود.

   این قسمت دارای پیچیدگی زمانی $O(n^2)$ است و بخش اصلی محاسبات شبیه‌سازی را تشکیل می‌دهد.
2. گرانش رو به پایین

   علاوه بر نیروی بین ذرات، یک نیروی ثابت رو به پایین نیز بر ذرات اعمال می‌شود.

3. مقاومت هوا
   برای شبیه‌سازی حرکت واقعی‌تر، اثر مقاومت هوا بر سرعت ذرات در نظر گرفته می‌شود.
4. برخورد با دیوار و زمین
   در صورت برخورد ذرات با مرزهای محیط شبیه‌سازی، سرعت آن‌ها با در نظر گرفتن اتلاف انرژی تغییر می‌کند.
5. عمر و رنگ ذرات
   هر ذره دارای طول عمر مشخصی است و با گذشت زمان عمر آن کاهش پیدا می‌کند. رنگ ذره نیز متناسب با وضعیت آن تغییر می‌کند.

ساختار محاسبات

برای هر ذره ابتدا نیروهای ناشی از سایر ذرات محاسبه می‌شود. برای محاسبه‌ی نیروی گرانشی، برای هر جفت ذره‌ی `i` و `j` فاصله‌ی آن‌ها محاسبه شده و سهم ذره‌ی `j` در شتاب ذره‌ی `i` به صورت زیر در نظر گرفته می‌شود:

```
None
F = (G * m_j) / (r^2 + SOFTENING)^(3/2)
a_x += dx * F
a_y += dy * F
```

که در آن:

- `G` ثابت گرانش است.
- `m_j` جرم ذره‌ی `j` است.
- `dx` و `dy` اختلاف مختصات دو ذره هستند.
- `r²` مربع فاصله‌ی دو ذره است.
- `SOFTENING` برای جلوگیری از مشکلات عددی در فاصله‌های بسیار کوچک استفاده می‌شود.

پس از محاسبه‌ی نیروها، سایر قوانین فیزیکی از جمله گرانش، مقاومت هوا، حرکت ذره و برخورد با مرزها اعمال می‌شوند.

وظایف

۱. بررسی ساختار پروژه

فایل‌های پروژه را مطالعه کرده و ساختار کلی برنامه را بررسی کنید. به‌خصوص موارد زیر را بررسی کنید:

- ساختار `Particle`
- ثابت‌ها و پارامترهای فیزیکی شبیه‌سازی
- نحوه‌ی مقدار دهی اولیه‌ی ذرات
- نحوه‌ی محاسبه‌ی نیروها
- نحوه‌ی اعمال قوانین فیزیکی
- نحوه‌ی نمایش ذرات روی صفحه

۲. پیاده‌سازی منطق شبیه سازی

این شبیه سازی باید شامل مراحل زیر باشد:

الف) محاسبه‌ی نیروهای N-body

برای هر ذره، اثر تمام ذرات دیگر را محاسبه کنید. توجه داشته باشید که یک ذره نباید نیروی گرانشی ناشی از خودش را در محاسبات وارد کند.

ب) اعمال قوانین فیزیکی

پس از محاسبه‌ی نیروها، قوانین فیزیکی شبیه‌سازی را روی ذرات اعمال کنید، از جمله:

- گرانش رو به پایین
- مقاومت هوا
- به‌روزرسانی سرعت
- به‌روزرسانی موقعیت
- برخورد با دیوارها
- برخورد با زمین
- کاهش عمر ذرات
- تغییر رنگ ذرات

نسخه‌ی Scalar باید رفتار صحیح و مطابق با منطق تعریف‌شده‌ی شبیه‌سازی داشته باشد.

۳. تکمیل حالت‌های اجرای شبیه‌سازی

همچنین نوع `ComputeMode` باید حالت‌های مختلف اجرای شبیه‌سازی را مشخص کند:

```C/C++
enum class ComputeMode : uint8_t {
    SCALAR = 0,
    SIMD   = 1,
    OPENMP = 2,
    CUDA   = 3
};
```

همچنین تابع `update_particles` را به گونه‌ای تکمیل کنید که بر اساس حالت انتخاب‌شده، تابع محاسباتی مربوطه را فراخوانی کند. در این قسمت تنها پیاده‌سازی حالت `SCALAR` مورد نیاز است و سایر حالت‌ها در بخش‌های بعدی پروژه پیاده‌سازی خواهند شد.

برای Build کردن پروژه در یک سیستم لینوکسی ابتدا پکیج های **libsdl2-dev libsdl2-ttf-dev** را نصب کرده و سپس پروژه را با **makeFile** داده‌شده Make کنید به عنوان مثال در سیستم عامل ubuntu:

```Shell
sudo apt install build-essential make pkg-config libsdl2-dev
libsdl2-ttf-dev

make

./build/simulation
```

در صورت استفاده از سیستم عامل های دیگر با مراجعه به داک های رسمی SDL و کامپایلر خود پروژه را بیلد کنید، ممکن است مجبور به تغییر MakeFile یا CMakeLists داده شده شوید.

ارائه و ارزیابی

برای این پروژه نیازی به تهیه و تحویل گزارش نیست.

ارزیابی پروژه به صورت ارائه‌ی ترجیحا حضوری انجام خواهد شد. شما باید پروژه را اجرا کرده و بتوانید بخش‌های پیاده‌سازی‌شده را توضیح دهید.

همی کد ها و اسکریپت های لازم اجرا را در یک فایل .zip آپلود فرمایید.

نکات و نحوه تحویل

- کد پروژه باید به زبان C/C++ نوشته شده و قابل کامپایل و اجرا باشد.
- در این قسمت، تنها پیاده‌سازی نسخه‌ی Scalar مورد نیاز است و نیازی به پیاده سازی الگوریتم با روش‌های موازی مانند OpenMP، SIMD یا CUDA نیست.
- نسخه‌ی Scalar باید مرجع صحیح شبیه‌سازی باشد؛ بنابراین صحت محاسبات آن نسبت به سرعت اجرا اولویت دارد.
- از تغییر رابط‌های مشخص‌شده در فایل‌های اصلی پروژه خودداری کنید.
- کد کامل پروژه را طبق روش اعلام‌شده در صفحه‌ی درس تحویل دهید.
- تنها یکی از اعضای گروه پاسخ تمرین را بارگذاری کند.
- در زمان ارائه، حضور تمام اعضای گروه الزامی است؛
- ممکن است در زمان تحویل از بخش هایی جز بخش هایی که باید کد آنها را بزنید سوال پرسیده شود.
- ممکن است به صورت تصادفی از هر یک از اعضای گروه درباره‌ی قسمت‌های مختلف کد سوال پرسیده شود.
- در صورت داشتن سؤال می‌توانید از طریق ایمیل یا تلگرام با طراحان تمرین در ارتباط باشید.

هدف این تمرین یادگیری شماست؛ لطفاً تمرین را خودتان انجام دهید. در صورت محرز شدن تقلب، مطابق با سیاست‌های درس برخورد خواهد شد.

موفق باشید